# Smaller proofs for K215, a checked marker transfer, and the 214-state frontier

Research note prepared for the ProveIt repository

October 21, 2026

**Abstract**

We inspect the current 215-state consistency-machine development and supply two concrete proof improvements. First, K215's source-correspondence theorem is rechecked in Rocq using an ordinary bisimulation certificate: all 2,240 signed offsets in the old certificate are zero, one direction of checking suffices by a generic symmetry theorem, and an exact positive-integer path encoding reduces the certificate source plus its decoder from 1,499,170 to 173,003 bytes. The new source theorem and the equality between decoded and original certificates are kernel checked.

Second, we identify the already submitted M124 marker machine as an especially inexpensive additional candidate to certify after K215. New independent Lean and Rocq proofs establish equivalence of their exact native transition tables on every initial binary tape from state zero, using two predecessor-marker invariants and one-or-three-step simulation. M124 still has 215 working states. Its existing machine construction is attributed to the DR215 submission; the checked short proofs are new here.

No valid 214-state machine is produced. The packet instead provides a finite inductive certificate for reachable register values, three justified source prunings, a complete obstruction to ordinary read-input control quotients of M124, and explicitly bounded local-search results. We distinguish these results from a state-count lower bound and from a fully integrated new ZFC endpoint.

## Contents

**1 Answer, scope, and the pinned baseline** **2**

**2 Shortening the existing proof without changing its theorem** **3**

**3 Which other 215-state candidate is easiest to certify?** **6**

**4 The 214-state search: proved restrictions and bounded failures** **9**

**5 A practical integration and research plan** **11**

**6 Artifacts, reproduction, and evidence boundaries** **12**

# 1 Answer, scope, and the pinned baseline

The most useful immediate actions are to replace the remaining offset-specific Rocq source proof by the ordinary zero-offset proof supplied here, and to certify M124 by a short local transfer from the already established K215 endpoint. There is no reason to re-prove the numerical calculus or its complete binary runtime for that additional machine.

The analysis is pinned to ProveIt commit

`e0b8f9cb8c496ff2d6e43f478021fc14d42bbfad`.

The primary machine lives under `Computability/BusyBeaver/Research/ZFC/candidate-k215/`; the proof development and its verification records live under `Computability/BusyBeaver/Research/ZFC/k225-formal/`. These primary sources, rather than older research summaries, determine the baseline [1, 2, 3].

## 1.1 Machine convention

All state counts in this report count working states, including initialization, and exclude a separate halt. The machines have one two-way infinite tape, two symbols with blank symbol zero, a blank initial tape for the consistency statement, and only left or right head moves. Native labels are sparse: a label as large as 302 does not imply 303 working states. Dense versions of K215 and M124 use working labels $0, \ldots, 214$ and halt label 215. Both native versions use initial label 0 and halt label 254.

| Artifact | SHA-256 |
|---|---|
| K215 native | `9683d6ed9628e6ce407c315b08f5da3d063da94e92706f275ca0f33e8272e4b2` |
| K215 dense | `fdbcef0825886035c3091c8537a69535bc96f9e265f0ae45863543d40ae70090` |
| M124 native | `8e0ccb4b631c4e4c1c02c4b52a6fe382984e67557dd1dcd949065479e38d07b9` |
| M124 dense | `fa15e120366da90e74aac43f3b808b27c0187960d373d4a95e425f8c73259e2b` |

## 1.2 What is already proved upstream

The repository records full Lean and Rocq proofs of the primary K215 endpoint. The original fresh builds checked 204 Lean project modules and 237 Rocq sources; subsequent proof simplifications reduce the current Lean closure to 185 project modules. The recorded upstream toolchains are Lean 4.32.0 and Rocq 9.1.0. Lean's endpoint assumptions are the standard `propext`, `Classical.choice`, and `Quot.sound`; the four Rocq endpoint assumption reports are closed [2].

The checked endpoint is an external proof-assistant statement of the form

$$\text{Halts}(K215, \text{blank}) \quad \Longleftrightarrow \quad \text{ZFC has a finite derivation of falsity.}$$

Its negation gives nonhalting equivalent to syntactic consistency. The repository separately presents an ordinary mathematical calibration establishing the requested equivalence in ZFC. Neither the old endpoint nor this packet is an unconditional proof of nonhalting. We do not construct an internally encoded PA or ZFC derivation in this investigation.

The present work does not repeat the full 185- or 237-module build. It freshly checks the new source-correspondence proof and its necessary extracted dependencies, the exact certificate identity, and both standalone M124 transfer proofs. Integrating replacements into the complete endpoint still calls for the repository's end-to-end build and assumptions audit.

### 1.3 What is new, inherited, and unfinished

| Item | Status in this packet |
|---|---|
| Primary K215 consistency calibration | Inherited from the pinned repository; unchanged. |
| Zero-offset source proof | New kernel-checked proof of an existing source-equivalence theorem. |
| Compact source certificate | New lossless representation; decoded equality to the original is kernel checked. |
| M124 machine construction | Inherited from the DR215 marker appendix. |
| M124 all-tape transfer | New independent Lean and Rocq proofs for the exact native tables. |
| M124 full ZFC endpoint | Ordinary transfer follows from the inherited calibration; the formal configuration-model bridge is not yet integrated. |
| Reachable-value invariant | Finite inductive certificate with an independent Python checker and a mathematical soundness proof; no new proof-assistant formalization claimed. |
| 214-state machine | Not obtained. No minimum-state theorem is claimed. |

## 2 Shortening the existing proof without changing its theorem

### 2.1 Avoid repeating work already completed

The current repository already shares guarded implication arguments, composes the AX routine from a finite word of macro contracts, uses a general enumerator theorem for both starts, proves enumerator block contracts by symbolic execution, factors site coverage, and derives reverse Lean offset certificates by symmetry [3]. Recommendations simply to "use reflection" or "share register lemmas" would therefore miss the present state of the development.

The remaining improvement below is specific: the current Rocq `Source215.v` still evaluates both directions of an offset-bisimulation certificate, although every offset is zero. Its large explicit path data also admit a lossless representation much smaller than lists of spelled-out Booleans.

### 2.2 The exact source theorem

Let $p_{215}$ be the existing register source and let $d_{\mathrm{Perm}}$ be the existing command-permuted program. For every natural-valued register file $R$, the new module proves

$$\mathrm{Halts}_{\mathrm{RM}}(p_{215}, ([], R)) \quad \Longleftrightarrow \quad \mathrm{Halts}_{\mathrm{RM}}(d_{\mathrm{Perm}}, ([], R)). \tag{1}$$

This is the same mathematical interface used by the endpoint. In the delivered Rocq code it is

```
Theorem p215_dagPerm_simple R :
  rm_halts p215 ([],R) <-> rm_halts dagPerm ([],R).
```

The definitions of both programs are the repository's literal definitions. They are neither substituted by new abstract parameters nor assumed equivalent. The forward certificate check is evaluated against these definitions.

### 2.3 Erase offsets that carry no information

The old certificate has 2,240 ordered entries, each containing two paths, a signed offset, and two successor indices. All 2,240 offsets are zero. For this particular pair of programs, related segment runs therefore leave equal register files, rather than register files related by an affine adjustment to one component. Related guarded tests read the same register.

Consequently the existing ordinary checker `K230P.Bisim230` suffices. The new proof does not import the signed-offset checker. This removes a 177-line, 7,886-byte offset-specific layer from this source theorem's dependency path. The number refers to that layer, not the whole transitive project closure.

**Proposition 2.1** (One forward certificate suffices)**.** *For the ordinary register-segment checker, if a certificate $C$ checks from tree $A$ to tree $B$, the certificate obtained by swapping each pair of related paths checks from $B$ to $A$.*

*Proof.* Inspect the finite certificate conditions. Related sites use the same tested register; swapping their order preserves that equality. Matching linear segments have equal increment counts for each register, an equality preserved by reversal. A branch ending at the two paths named by a successor entry ends at the swapped paths in the mirrored entry. Halting sites remain paired with halting sites. These observations prove the entry and branch conditions for every mirrored row and for the initial branch. The generic checker theorem then yields the reverse check. The existing checker soundness theorem supplies both directions of (1). □

The generic argument is compiled as `BisimMirror.v`. A separate `OffsetMirror.v` proves the analogous result for genuine signed offsets: swapping a pair negates its offset, and the segment-count equation is preserved by negating its two sides. That more general theorem is useful for other candidates but is not a dependency of the new K215 source theorem.

The resulting source interface is 21 lines, 759 bytes. Its central computation is a single forward check. We measured a successful local checker evaluation, but do not infer any total endpoint speedup: serialization, imported dependencies, and kernel rechecking have separate costs.

### 2.4 A lossless path codec

Encode a finite bit list by a positive integer:

$$e([]) = 1, \qquad e(0 :: p) = 2e(p), \qquad e(1 :: p) = 2e(p) + 1.$$

The inverse removes the least significant bit until the sentinel 1 is reached. For example, $e([0, 1]) = 6$, whereas $e([]) = 1$ and $e([0]) = 2$. The sentinel preserves length even when a path ends in zero bits.

**Lemma 2.2** (Exact path representation)**.** *For every finite bit list $p$, decoding $e(p)$ returns $p$. For every positive integer $u$, re-encoding its decoded path returns $u$.*

*Proof.* Use structural induction on lists for the first assertion and structural induction on the binary positive-number constructors for the second. The two nonterminal cases remove precisely the constructor just added by the encoder. □

The codec proof is `PathCodec.v`. The more important data-specific theorem in `Cert215Exact.v` states that expanding the entire compact certificate gives exactly the original ordered certificate, including all paths, zero offsets, and successor indices. Thus the compression has a checked equality behind it; a checksum alone is not being substituted for a logical argument.

| Uncompressed UTF-8 source | Bytes |
|---|---|
| Original `Cert215Offsets.v` | 1,499,170 |
| Compact data | 171,678 |
| Reusable codec | 1,325 |
| Compact data plus codec | 173,003 |

The last quantity is 88.4601% smaller than the original certificate. It does not measure the size of the complete proof, compiled objects, or the trusted kernel. The generated file still has roughly one row per certificate entry; the improvement is in bytes and representation, not a claim to remove 88.5% of all proof lines.

### 2.5 Checked implementation and compatibility

The source theorem, generic mirror lemmas, and exact certificate equality compile under Coq 8.18.0 with OCaml 4.14.1. Their printed assumptions are closed. The local compiler predates the repository's Rocq 9.1.0. The extracted dependency closure therefore includes a small compatibility module providing renamed list-library lemmas, all proved without added axioms. The original 9.1 source is retained unchanged beside the private compatibility copy. The build driver has a 9.1 branch, but that branch was not run here.

The new source theorem has no dependency on the optional signed-offset module. The exact-expansion audit is deliberately separate so that routine source checking need not import the original long certificate after integration.

An optional recursive check of the initial transparent-expansion source-proof implementation was stopped after 409.2 seconds without completion. That interrupted run is preserved, and no successful recursive check of the entire new source closure or complete ZFC endpoint is claimed here. The final compact definition materializes its expanded data at definition time, while the exact-expansion equality remains checked. Compiler acceptance, closed assumption reports, and the separate recursive exact-certificate check are recorded individually; source-size reduction must not be mistaken for a measured whole-build acceleration.

### 2.6 Further refactors with concrete prerequisites

**Lean decoder bound.** Lean already supplies ordinary bisimulation symmetry, but its ordinary path decoder stops at a smaller bound than K215 needs. The two maximum path lengths in this source comparison are 45 and 176 bits. The offset checker currently uses a 512-step decoder; the ordinary checker uses a recursion fuel of 128. A migration must parameterize the bound or supply a sufficiently large ordinary-decoder instance. Merely changing the certificate type from offset to ordinary is not a correct patch.

**Separate encodings from old program data.** Rocq's `Prog215.v` imports the 149,114-byte `Prog230.v` to reuse generic `cedge` and `cv` definitions near its beginning. Move these into a small encoding module and audit clients before dropping the old program import. This may avoid bringing the unrelated 5,124-node K230 program and its well-formedness proof into the relevant closure. This refactor is proposed, not implemented or measured as an endpoint reduction here.

**Index certificate successors efficiently.** The Rocq checkers use `nth_error` on a list to retrieve successor entries. For $N$ certificate rows with a constant number of successor queries per row, this contributes $O(N^2)$ worst-case list-indexing work. A verified balanced-tree representation can reduce

that component to $O(N \log N)$, separately from path decoding and segment execution. Lean already uses balanced certificate lookup. Porting its representation, or a generic verified finite-map interface, is a concrete next checker improvement; no measured whole-proof speedup is claimed here.

# 3 Which other 215-state candidate is easiest to certify?

## 3.1 The relevant distinction: an additional proof versus a fresh foundation

After the verified K215 endpoint is available, an alternative can be certified by proving equivalence to K215. The candidate-specific proof can be small even though the inherited transitive foundation is substantial. A shorter additional proof does not mean the old calculus, compiler, and runtime development vanishes.

Under this incremental criterion, the existing **M124** candidate is the strongest recommendation found. It occurs in the marker appendix of `docs/incoming/DR215_research_2026-10-21.zip`, at member directory `DR215_research_2026-10-21/markers/marker_family/m124/` [5]. M160 in the same family has a similarly local structure, but the new proofs in this packet target M124 only.

| Candidate | Additional obligations after primary K215 |
|---|---|
| M124 / M160 | Four changed rows, two immediate-predecessor marker facts, two three-step blocks. |
| CM215 | Six changed rows; a two-step history exception supplements its predecessor markers. |
| Context-return variant | Several distinct local history-dependent blocks. |
| commands120 / composed / R41 | Much high-level mathematics is inherited, but a changed physical decoder or allocation still needs a larger runtime transfer. |
| NW215 | Four physical rows conceal 352 changed source register operands and a register/tape correspondence. |
| ZC215 | An attractive tag-lowering injection simplifies a scheduler argument, but refill and startup change. |
| ST / DR / GC / FT / SB | Compiler, scheduler, protected-tail, or stack-credit interfaces change; these require more than a small transition-table comparison. |

This is an engineering assessment of the inspected obligations, not a theorem about shortest proof lengths. In particular, FT215 changes one scheduler row but requires the forced-transfer debt-seven argument. Row count by itself is a poor proxy for proof difficulty. The original K215 remains the inspected candidate whose complete existing ZFC endpoint has already been integrated in both proof assistants.

## 3.2 The four-row M124 transformation

Only four native rows change:

| State / read | K215 | M124 |
|---|---|---|
| 110 / 0 | 0 $R$ 127 | 0 $R$ 124 |
| 124 / 0 | 0 $R$ 62 | 0 $L$ 127 |
| 127 / 0 | 0 $R$ 14 | 0 $R$ 8 |
| 127 / 1 | 1 $R$ 138 | 1 $R$ 59 |

Both tables have 215 working states and 430 rows. All targets are working labels or the separate halt. Their dense/native bijection is included. M124 is not just a relabeling of K215: it has 83 left-moving rows, versus K215's 82. This invariant also rules out an isomorphism obtained by global tape reflection, which would interchange the old counts 82 and 348.

Let a configuration be $(q, h, t)$ with $q$ a native control, $h \in \mathbb{Z}$, and $t : \mathbb{Z} \to \{0, 1\}$. Define

$$\phi(q) = \begin{cases} 124 & q = 127, \\ q & \text{otherwise}, \end{cases} \qquad E(q, h, t) = (\phi(q), h, t).$$

The map is intentionally not a bijection at simulation boundaries. Old roles 124 and 127 share new control 124; physical label 127 is reused inside the new selector block.

### 3.3 Two invariants replace a large runtime recertification

**Lemma 3.1** (Predecessor markers)**.** *Every K215 configuration reachable from control 0, on any initial tape and at any initial head position, satisfies*

$$q = 124 \Longrightarrow t(h-1) = 1, \qquad q = 127 \Longrightarrow t(h-1) = 0. \tag{2}$$

*Proof.* The only transition entering old 124 is $104/1 : 1\ R\ 124$. It writes one at the previous head and moves right, leaving one immediately left of the new head. The only transition entering old 127 is $110/0 : 0\ R\ 127$, which similarly leaves zero. The initial control is neither exceptional state. Induction over transitions proves both implications; no fact about the rest of the tape is required. The exclusive predecessor census is checked against the complete literal table. □

**Lemma 3.2** (Positive local simulation)**.** *Suppose an old configuration $c$ satisfies (2). Its K215 step is simulated from $E(c)$ by one or three M124 steps, ending at exactly $E(c')$, with the same complete tape and head as the old successor $c'$. Three steps are needed precisely for old states 124 or 127 reading zero.*

*Proof.* For old 124 reading zero, the left marker is one. The new computation is

$$(124, h, t) \longrightarrow (127, h-1, t) \longrightarrow (59, h, t) \longrightarrow (62, h+1, t).$$

Each write reproduces the symbol already present. The final configuration is the old successor. For old 127 reading zero, its image starts at 124 and the marker is zero:

$$(124, h, t) \longrightarrow (127, h-1, t) \longrightarrow (8, h, t) \longrightarrow (14, h+1, t).$$

Again the entire tape is preserved. When either old role reads one, both old transitions and the new boundary transition use $1\ R\ 138$. All remaining rows commute with $\phi$, including the redirected incoming row 110/0. The finite-table certificate checks that last universal assertion. □

### 3.4 Why the local simulation proves both halting directions

**Theorem 3.3** (All-tape equivalence)**.** *For every binary tape $t$ and every head position $h$,*

$$\text{Halts}(K215, (0, h, t)) \quad \Longleftrightarrow \quad \text{Halts}(M124, (0, h, t)).$$

*Proof.* Let $c_n$ be the old configuration after $n$ transitions, extending a halted computation by the identity. Inductively concatenate Lemma 3.2's blocks. There are matching new times $s_n$ with

$$n \le s_n \le 3n, \qquad d_{s_n} = E(c_n).$$

More precisely, $s_n = n + 2z_n$, where $z_n$ counts exceptional old zero-reading steps among the first $n$ steps. The initial marker invariant is Lemma 3.1, and the old step preserves it.

If the old computation halts at time $n$, its new match at $s_n$ is also at the halt, because $\phi$ neither changes the halt nor maps any other state to it. Conversely, suppose the new computation halts at time $k$. Its halt is absorbing in the extended execution. Since $s_k \ge k$, the matching boundary $d_{s_k}$ is still halted. Therefore $E(c_k)$, and hence $c_k$, is halted. Positivity of every block is essential to this converse; a zero-length simulation would not justify it. □

The proof is elementary enough to formalize in ZFC. Composing it with the inherited mathematical K215 calibration yields

$$\mathrm{ZFC} \vdash (\neg\,\mathrm{Halts}(M124, \mathrm{blank}) \leftrightarrow \mathrm{Con}(\mathrm{ZFC})),$$

with the same interpretation of consistency as the parent calibration. This is a transfer of that calibration, not a new independently constructed foundational theory.

### 3.5 What the proof assistants actually checked

The new Rocq development comprises 224 lines of generated table data, 96 lines of finite facts, and 220 lines of semantics, invariant, simulation, and halting arguments. Coq 8.18.0 compiles all three modules. Its three printed assumption reports are closed, and recursive `coqchk -silent M124Transfer` succeeds.

The new independent Lean development uses a head-centered integer tape. Its exact-table certificate checks predecessor facts, regular-row commutation, working-label distinctness, the 215-state count, and closure of both transition tables on working states plus halt. Lean 4.32.0 checks `Marker124.all_tapes` and `Marker124.blank_nonhalting`; the reported axioms are only `propext`, `Classical.choice`, and `Quot.sound`. There are no admitted local lemmas.

Rocq avoids functional extensionality by reusing the tape function on same-symbol writes, then proves that this implementation agrees pointwise with ordinary tape update. Lean uses ordinary function equality and its standard extensionality principles. The mathematical transition systems are the usual full binary-tape systems.

**Formal integration boundary.** These are closed table-to-table theorems. They do not yet import the repository's complete ZFC endpoint into their new tape models. The existing Rocq endpoint uses BusyCoq stream zippers and a legacy sparse numbering that maps native 301 to 102, native 302 to 120, and native halt 254 to 230. A bridge must respect these maps. The smallest integration patch is likely to port the short marker proof directly to BusyCoq's step relation; alternatively prove a reusable stream-zipper/integer-tape bridge. Lean has a comparable zipper-based interface. The large parent proof is then reused as a dependency.

The supplied proposition-polymorphic Rocq adapter makes the remaining dependency explicit. It does not introduce an axiom named "K215 is correct" or silently assume the tape-model bridge. Consequently we do not describe this packet as a fully integrated new ZFC endpoint in either assistant.

### 3.6 Initialization is transferred, but its duration changes

An independent finite replay checks every synchronization boundary in the parent's complete 95,714-step bootstrap. There are 63 exceptional old steps: 33 at 124/0 and 30 at 127/0. Thus M124 reaches the exact same calibrated terminal configuration after

$$95,714 + 2 \cdot 63 = 95,840$$

steps. The terminal native control is 84, the head is at $-8$, and the complete tapes agree, with 95 one-cells. This replay is included as a finite validation of the literal tables. The infinite result rests on Theorem 3.3, not on extrapolation from the replay.

## 4 The 214-state search: proved restrictions and bounded failures

### 4.1 A complete obstruction to ordinary control quotients

M124's role merger is state neutral: a freed boundary role pays for its selector. Could a direct control-state quotient remove another label? Under the usual output-preserving, unrestricted-read-input congruence, the answer for this exact table is no.

Call an equivalence relation on controls a read-input congruence if it separates the halt from working states and, whenever $q \sim r$, for each read bit $b$ the two transitions write the same bit, move in the same direction, and have equivalent destinations. Every direct quotient with identical per-step actions must satisfy this condition.

**Proposition 4.1.** *Every read-input congruence on M124's 215 working controls and separate halt is equality.*

*Proof.* The packet supplies a distinguishing read word for each of the $\binom{215}{2} = 23,005$ unordered working-state pairs. Each word causes either different write/move outputs or different halt status. Every witness has length at most 10, and a separate evaluator replays every witness against the exact table. Congruent states must have the same outputs and halt status under every finite read word, by induction on the word length. Thus none of these pairs can be congruent. □

This is an exact finite obstruction to a specific reduction method. A freely chosen read word need not occur on a real tape. Proposition 4.1 is therefore not a lower bound for blank-input Turing machines, not a characterization of all-tape equivalence, and not an obstruction to invariants such as the markers used above.

### 4.2 An inductive reachable-register certificate

The accepted source starts with nine logical registers

$$(4, 0, 0, 0, 0, 0, 0, 0, 0).$$

For each source ordinal and register, we compute a subset of the three atoms

$$A_0 = \{0\}, \qquad A_1 = \{1\}, \qquad A_2 = \{2, 3, 4, \ldots\}.$$

Each program point denotes the product of its nine recorded sets; no hidden correlation between registers is assumed. The independent checker derives control destinations separately from the generator, verifies inclusion of the initial tuple, and verifies every possible abstract successor.

Increment maps $A_0$ into $A_1$, and $A_1, A_2$ into $A_2$. A positive decrement maps $A_1$ into $A_0$ and $A_2$ into $A_1 \cup A_2$; the zero branch uses only $A_0$. These exact transfers justify the finite test even for arbitrarily large register values. Representatives 2 and 3 realize all abstract decrement outcomes of $A_2$; checking them is not an assumption that values are bounded by 3.

**Theorem 4.2** (Soundness of the reachable-value certificate)**.** *Every finite source execution from the calibrated initial tuple lies in the product set certified at its current ordinal.*

*Proof.* The initial inclusion is explicitly checked. Suppose the claim holds at a source point. The executed instruction is increment, guarded decrement, or a control transfer. The exact abstract operation includes its concrete effect; untouched components remain in their previous sets. The checked edge inclusion then places the concrete successor in the product set at its destination. Induction over finite executions proves the claim. □

The independent checker validates 6,170 abstract edges and 4,194 point/register facts asserting zero. In particular:

| Ordinal | Instruction | Certified fact |
|---|---|---|
| 4671 | `CD 1` | Register 1 is zero; positive branch impossible. |
| 4672 | `CI 6` | Entire ordinal unreachable. |
| 4673 | `CC 2` | Entire ordinal unreachable. |
| 4733 | `CD 6` | Register 6 is zero; positive branch impossible. |
| 4734 | `CC 2` | Entire ordinal unreachable. |

This eliminates three whole source sites and five guarded cases from the reachable source execution. It does not eliminate them from arbitrary register frames or arbitrary initial binary tapes. The ordinary proof plus checker is an unbounded invariant argument, but it is not newly kernel formalized here.

### 4.3 Why dead-code removal did not immediately save a state

The invariant justifies removing the no-op interval $[4671, 4674)$, the interval $[4733, 4735)$, or both. The pruner checks the old/new ordinal map and retained guarded edges. At an eliminated interval, its reachable execution is one identity step followed by exit; deleted positive branches are impossible. There is no infinite run made solely of deleted steps, so the source halting equivalence holds in both directions.

| Source | Primitive occurrences | Distinct subtrees |
|---|---:|---:|
| Accepted K215 | 4,751 | 202 |
| First interval removed | 4,748 | 203 |
| Second interval removed | 4,749 | 204 |
| Both removed | 4,746 | 205 |

All three variants retain maximum depth 47 and maximum zero run 6. Deleting instructions destroys existing subtree sharing. The physically important description has therefore become larger despite a shorter expanded instruction word. A time-capped reassociation beam inspected 13,097 distinct trees for the first variant and retained minimum subtree count 203; this is not a minimum theorem.

The inherited runtime row inventories for the surviving cases still cover all 430 physical rows. Dropping those five source cases alone does not expose a provably unused physical transition. The next constructive step must coordinate semantic pruning with allocation or with a more precise binary invariant.

### 4.4 Twelve frozen-table context searches

A fresh search tested 947,720 local fold specifications across twelve frozen final tables. The scope was predecessor contexts of depth five, at most 97 literal transitions per symbolic block, and at most 512 symbolic nodes. Unknown tape cells branch symbolically, and a successful block would have to reach the exact mapped destination with the exact head displacement and full visited-tape effect. No specification survived.

These are twelve literal tables, not twelve distinct transition-graph isomorphism classes: two composed presentations are related by a state bijection. Some specifications also become identical after redirection. The count is a count of tested specifications, not a count of distinct machines. The supplied summary records exact hashes and initial controls; in particular, zero-spare tables start at 74 rather than an assumed 0.

Resource-limit exits are unresolved attempts, not counterexamples. An earlier, weaker R41-only probe is retained for provenance but is not added to the main total as a disjoint search.

### 4.5 Selective deletion of M124's selector

We also delete new helper 127 and vary only row 124/0. There are 214 remaining destination labels, two write bits, and two directions, hence exactly 856 replacements. Each must satisfy both marker contracts: finish at 14 for marker zero and at 62 for marker one, one cell to the right, restoring the tape, with arbitrary outside cells. The original helper is checked first as a positive sanity control.

No replacement passed both contracts within the bounds of 96 subsequent transitions, ten visited cells, and 4,096 symbolic nodes per case. Of the first-marker attempts, 851 hit the cell cap, one hit the step cap, three halted or entered a missing state, and one passed. The last candidate hit the cell cap on the second marker. The predominance of bound exits is a reason to describe this as failed bounded certificate search, not an impossibility proof.

## 5 A practical integration and research plan

### 5.1 Changes ready for a focused source-proof patch

1. Add the path codec, compact data, and ordinary mirror theorem under the project namespace. Retain the original certificate only in a separate exact-expansion audit target.

2. Replace the two direction-specific computations in `Source215.v` with the single forward plain check and `check_one_sided_iff`. Preserve the public source-equivalence theorem name.

3. Rebuild `Correct215` and `CorrectDense215`, then print their assumptions. The local source theorem is already checked; this final step tests integration with the complete endpoint.

4. Address Lean's decoder bound before making the analogous zero-offset specialization. Do not change its bound silently inside generated data.

### 5.2 The shortest credible next endpoint

Port the M124 marker invariant and two three-step blocks directly into the existing tape semantics. Reuse a single positive-block halting-transfer theorem rather than copying the global time argument for every future local transformation. Bind the complete literal tables and label maps by finite computation, then compose with K215's existing endpoint. This creates a small, reviewable change while maintaining the established foundation.

Candidate selection should track three distinct costs: new semantic assumptions or invariants, new binary simulation obligations, and serialized certificate size. A candidate with fewer primitive operations may have a larger shared source and a harder tape invariant; a one-row patch can require a new infinite-schedule theorem. M124 performs well on the first two costs because the global logical program is unchanged.

### 5.3 Further research questions

1. **Can the marker selector share a state only in a certified phase?** Search for a joint transformation in which another role frees a row or control for the selector. The exact contracts at old 124 and 127 provide a small verification target; restricting all change to a single replacement row was too narrow in the tested class.

2. **Can register-value invariants be transported to decoder controls?** The product-domain source certificate is cheap and sound. A useful next certificate would relate each reachable source phase to selected nearby tape cells, making physical transitions removable without requiring an all-register-frame theorem.

3. **Can pruning and source sharing be optimized jointly?** The three dead-code deletions worsen subtree counts. Search over source semantics and shared decomposition together, using physical allocation cost as the objective rather than primitive-word length alone.

4. **What is the smallest reusable local-transfer interface?** Marker history, explicit positive block length, exact tape update, and halt reflection are sufficient here. A verified finite checker for such local contracts could make future 215- and 214-state submissions much cheaper to assess.

5. **Can one certificate generator feed both proof assistants?** Use a small language-neutral certificate schema, independently decoded and checked by each kernel. The generator should remain untrusted; schema alignment and literal data binding must be checked.

6. **How much dependency mass disappears after extracting generic encodings?** The `cedge/cv` import is a concrete opportunity. Measure the transitive endpoint closure and final kernel work after refactoring rather than treating source bytes as build time.

7. **Can the cleaner zero-credit scheduler replace the current credit argument economically?** ZC215's tag-lowering injection is promising for a future foundation-level rewrite, but its proof cost should include changed initialization and physical runtime. A comparison of complete obligations is needed before choosing it over the current verified endpoint.

## 6 Artifacts, reproduction, and evidence boundaries

The archive is self-contained for the new source proof, M124 local proofs, finite input checks, replay, and saved search evidence. It does not bundle the entire 1.6 GB Busy Beaver checkout or the complete existing endpoint closure.

| Directory / file | Purpose |
|---|---|
| `source_proof/` | Compact certificate, exact-expansion audit, plain and offset symmetry, source theorem, extracted dependencies, licenses, and build records. |
| `m124_rocq/` | Exact native/dense machines, finite binding checker, three proof modules, assumption and kernel receipts, comparison and integration notes. |
| `m124_lean/` | Independently checked native-table facts and centered-tape transfer, deterministic generator, reproduction driver, and build receipt. |
| `research214/` | Independent invariant checker, source prunings, frozen input tables, bounded search code, and compressed shared-tree evidence. |
| `checks/` | Every-boundary initialization replay and unrestricted-read control-quotient witnesses. |
| `verify.py` | Package hashes and finite reproducibility checks; proof builds are explicit separate commands. |
| `report.tex`, `report.pdf` | This editable report and its compiled version. |

Run the finite checks from a clean extraction with Python 3 and assertions enabled:

```
python3 -B verify.py
```

Rebuild the new proofs with the appropriate toolchain:

```
cd source_proof
python3 build.py

cd ../m124_rocq
python3 reproduce.py

cd ../m124_lean
python3 reproduce.py --lean /path/to/lean
```

The Lean driver uses the compiler supplied on its command line, in a fresh private build directory. The reported run uses Lean 4.32.0. The Rocq drivers use Coq 8.18.0 locally; compatibility and the untested 9.1 route are documented in `source_proof/README.md`. Saved raw logs and per-file hashes make the claimed checks concrete.

The bounded context searches can be rerun separately using the commands in `research214/README.md`; routine verification does not rerun every search. A time-capped beam can visit a different number of trees on another computer, so exact saved finalists are retained and checked independently. No timed search log is promoted to a universal negative theorem.

**Result.** There is a concrete smaller proof of K215's existing source equivalence, and a particularly simple additional 215-state candidate with newly checked local equivalence in both proof assistants. The new mathematical and computational evidence does not yet cross the 214-state threshold. The most focused next attack is a joint source-phase and selector-sharing transformation whose certificate can reuse the short local-transfer framework.

## References


[1] ProveIt, accepted K215 machine and proof exposition, pinned revision `e0b8f9cb8c49`. https://github.com/VladimirReshetnikov/ProveIt/blob/e0b8f9cb8c496ff2d6e43f478021fc14d42bbfad/Computability/BusyBeaver/Research/ZFC/candidate-k215/README.md.

[2] ProveIt, formal correctness and recorded Lean/Rocq verification of K215. https://github.com/VladimirReshetnikov/ProveIt/blob/e0b8f9cb8c496ff2d6e43f478021fc14d42bbfad/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/README.md.

[3] ProveIt, existing proof-simplification checkpoints through phase four. https://github.com/VladimirReshetnikov/ProveIt/blob/e0b8f9cb8c496ff2d6e43f478021fc14d42bbfad/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/README.md.

[4] ProveIt, index of materially distinct 215-state alternatives. https://github.com/VladimirReshetnikov/ProveIt/blob/e0b8f9cb8c496ff2d6e43f478021fc14d42bbfad/Computability/BusyBeaver/Research/ZFC/candidate-k215/alternatives/README.md.

[5] ProveIt, DR215 research archive, including M124's original marker construction, ordinary proof, and exact machine. https://github.com/VladimirReshetnikov/ProveIt/blob/e0b8f9cb8c496ff2d6e43f478021fc14d42bbfad/docs/incoming/DR215_research_2026-10-21.zip.

[6] ProveIt, exact Lean endpoint interface. https://github.com/VladimirReshetnikov/ProveIt/blob/e0b8f9cb8c496ff2d6e43f478021fc14d42bbfad/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/KEndpoint215.lean.